

3주차 - 반복문

📅 날짜 2026년 4월 6일

리스트와 반복문

리스트 : 여러가지 자료를 저장할 수 있는 자료 ([])

요소 (element) : 대괄호 내부에 넣는 자료

인덱스 : 대괄호 안에 들어간 숫자

```
list_a = [273, 32, 103, "문자열", True, False]
list_a[0]
list_a[1]
list_a[2]
list_a[1:3]

273
32
103
[32, 103]
```

- 리스트 사용법
 - 리스트 변경

```
list_a = [273, 32, 103, "문자열", True, False]
list_a[0] = "변경"
list_a

["변경", 32, 103, "문자열", True, False]
```

- 대괄호 안에 음수를 넣어 뒤에서부터 요소 선택

```
list_a = [273, 32, 103, "문자열", True, False]
list_a[-1]
```

False

- 리스트 접근 연산자 이중 사용

```
list_a = [273, 32, 103, "문자열", True, False]
list_a[3]
list_a[3][0]

'문자열'
'문'
```

- 리스트 안에서 리스트 사용

```
list_a = [[1,2,3], [4,5,6], [7,8,9]]
list_a[1]

[4,5,6]
```

리스트 요소 추가하기 : append(), insert()

append() : 리스트 뒤에 추가

insert() : 리스트 중간에 추가

```
list_a = [1,2,3]

print("리스트 뒤에 요소 추가")
list_a.append(4)
list_a.append(5)
print(list_a)
print()

print("리스트 중간에 요소 추가")
list_a.insert(0,10)
print(list_a)

리스트 뒤에 요소 추가
[1,2,3,4,5]
```

리스트 중간에 요소 추가
[10,1,2,3]

리스트 요소 삭제하기 : del(), pop()

del (삭제만 한다)

특정 위치의 값을 그냥 삭제

삭제된 값은 반환 안 함

기본 예제

```
numbers = [10, 20, 30, 40]

del numbers[1]

print(numbers)
```

출력

```
[10, 30, 40]
```

범위 삭제

```
numbers = [10, 20, 30, 40, 50]

del numbers[1:4]

print(numbers)
```

출력

```
[10, 50]
```

설명

- index 1부터 3까지 삭제됨

전체 삭제

```
numbers = [1, 2, 3]

del numbers
```

리스트 자체가 사라짐 (변수 없어짐)

pop() (삭제 + 값 반환)

값을 꺼내면서 삭제

시험에서 진짜 많이 나옴

기본 예제

```
numbers = [10, 20, 30]

x = numbers.pop()

print(x)
print(numbers)
```

출력

```
30
[10, 20]
```

마지막 요소가 빠짐

특정 위치 pop

```
numbers = [10, 20, 30, 40]

x = numbers.pop(1)

print(x)
print(numbers)
```

출력

```
20
[10, 30, 40]
```

del 과 pop 핵심 차이

구분	del()	pop()
기능	삭제만	삭제 + 값 반환
반환값	없음	있음
사용 상황	그냥 지울 때	값을 사용할 때

문제 유형 1

```
numbers = [1, 2, 3, 4]

x = numbers.pop()

print(x)
print(numbers)
```

정답

```
4
[1, 2, 3]
```

문제 유형 2

```
numbers = [1, 2, 3, 4]

del numbers[2]

print(numbers)
```

정답

```
[1, 2, 4]
```

문제 유형 3 (헛갈리는 거)

```
numbers = [10, 20, 30]

del numbers[1]

print(numbers[1])
```

정답

```
30
```

이유

- 20 삭제되면서 리스트가 당겨짐

for문

정해진 횟수만큼 반복할 때 사용

기본 구조

```
for 변수 in 반복가능한것:
    실행할코드
```

예제 1: 5번 반복하기

```
for i in range(5):
    print("안녕하세요")
```

출력

안녕하세요
안녕하세요
안녕하세요
안녕하세요
안녕하세요

range() 설명

```
range(5) # 0, 1, 2, 3, 4  
range(1, 6) # 1, 2, 3, 4, 5  
range(1, 10, 2) # 1, 3, 5, 7, 9
```

예제 2: 1부터 10까지 출력

```
for i in range(1, 11):  
    print(i)
```

예제 3: 리스트 반복

```
fruits = ["사과", "바나나", "포도"]  
  
for fruit in fruits:  
    print(fruit)
```

while문

조건이 참인 동안 계속 반복

기본 구조

```
while 조건:  
    실행할코드
```

예제 1: 1부터 5까지 출력

```
i = 1

while i<=5:
    print(i)
    i = i + 1
```

중요: i 증가 안 하면 무한 반복됨 (시험 자주 나옴)

예제 2: 입력 받을 때까지 반복

```
password=""

while password != "1234":
    password = input("비밀번호 입력: ")
```

for vs while 차이

구분	for	while
사용 상황	횟수 정해짐	조건에 따라 반복
예시	1~10 출력	비밀번호 맞을 때까지
특징	간단하고 안전	무한루프

자주 쓰는 패턴

합 구하기

```
total = 0

for i in range(1,6):
    total = total + i

print(total)
```

짝수만 출력


```
for i in range(1,11):  
    if i % 2 == 0:  
        print(i)
```

break / continue

break → 반복문 종료

```
for i in range(10):  
    if i==5:  
        break  
    print(i)
```

👉 0~4까지만 출력

continue → 건너뛰기

```
for i in range(10):  
    if i==5:  
        continue  
    print(i)
```

👉 5만 제외하고 출력